Text2Mem：统一记忆操作语言与状态验证

ngocnamd93-spec

(天津大学，第 8 队)

摘　要: 本文阅读 Text2Mem 的统一操作规范和基准，梳理十二操作、五字段 Schema 与校验、解析、适配流程，区分结构相似、执行成功和状态断言满足率。作者结果显示高可执行性并不保证预期状态效果。本文进一步讨论参考后端、复杂语义和长期任务的验证边界，并说明其与本组 StructMem 课设的联系。本文没有新增 Text2Mem 实验。

关键词: 记忆操作；结构化规范；状态验证；论文阅读
中图法分类号: TP18

Text2Mem: Unified Memory Operations and State Verification

ngocnamd93-spec

(Tianjin University, Team 8)

1　Text2Mem：记忆操作的统一表达与状态验证

Text2Mem 面向自然语言驱动的记忆控制，提出统一操作语言及其基准[1]。本文关注的问题是：用户要求更新、合并或调整记忆时，系统能否明确表达目标范围与参数，并验证执行后是否得到预期状态。以下页码均为仓库原论文 PDF 页码，区别于出版物页码 2105–2119。本文整理作者的方法与实验，并给出阅读分析；本组未运行 Text2Mem 模型实验。

1.1　研究问题与适用范围

长期交互不仅需要检索历史信息，还需要控制记忆的优先级、有效期、合并和删除。例如，“暂时不要再提午餐”可能意味着降低显著性，也可能被误解为删除记录。如果动作、范围与时间未被明确表达，系统即使调用成功，也可能改变错误的记忆。论文因此在自然语言与具体后端之间引入结构化操作规范，而不是提出新的向量检索算法（原文第 1–3 页，图 1）。

与直接生成 SQL 相比，中间规范用记忆生命周期操作表达意图，再由后端实现其语义。它为不同后端提供共同描述的方向，但不能把架构中的适配能力写成跨后端性能已得到充分验证；作者主实验明确限定为 SQL/SQLite 参考环境（第 5、7 页，§3.3、§4.3）。

1.2　操作集合、Schema 与执行流程

十二个操作。论文将操作分为编码、存储和检索三阶段。编码阶段仅有 Encode；存储阶段包括 Update、Label、Promote、Demote、Merge、Split、Delete、Lock、Expire 九个操作；检索阶段包含 Retrieve、Summarize。Encode 组织内容、语义、来源与治理属性，向量生成可以延迟，不能简单等同于计算嵌入。Promote 与 Demote 调整显著性，Demote 可以归档而不删除；Delete 处理软/硬删除，Expire 设置到期规则，三者具有不同语义（第 4 页表 1，第 11 页附录 A）。

五字段契约。操作由 `stage`、`op`、`target`、`args`、`meta` 描述，分别对应阶段、动作、目标、专属参数和执行上下文。不同操作的必填条件不同，例如 Encode 创建新记忆时不必先选择已有目标。对于需要定位已有条目的操作，`target` 在 `ids`、`filter`、`search`、`all` 四种选择方式中择一；跨字段约束限制锁定、到期和大范围写入等行为（第 4–5 页，§3.2）。

Validator–Parser–Adapter。Validator 检查结构完整性与跨字段约束；Parser 把合法实例转换为类型化操作，归一化时间和优先级等参数；Adapter 再映射为具体后端动作。执行返回 `ExecutionResult`，记录受影响条目和状态

变化。多步骤请求可表示为保留依赖顺序的 `schema_list`（第 3 页图 2、第 5–7 页 §3.3–4.1 及图 3）。这个流程能检查格式和部分治理约束，但不能只靠合法 JSON 证明模型选对了目标或正确理解用户意图。

教学案例：提高课设进度记忆的优先级。参照附录 B.1.1 的 OKR 示例，构造请求“提高本周课设进度笔记的重要性，其他记录不变”。在目标 ID 已明确的简化情况下，可用 STO 阶段的 Promote，目标为 `ids=[17]`，参数为 `weight=0.9`。这只是未执行的教学实例，不是本组实验。检查应同时覆盖：条目 17 的权重是否达到要求、其内容是否未被误改、其他条目是否保持不变。仅返回执行成功，不能回答后两项；若目标选择阶段就选错条目，格式合法的操作仍可能产生错误结果（第 12 页附录 B.1.1；后两项为本文的检查建议）。

1.3 基准、指标与主要结果

基准以自然语言指令、前置操作建立的初态、参考 Schema 与 SQL 状态断言组成评测实例，区分中英文、直接/间接指令和单操作/工作流。作者比较 GPT-4o、Qwen2.5-72B-Instruct、Claude-4-Sonnet、Gemini-2.5-Pro 与 GPT-5，并给出相同自然语言指令与提示模板下的结果（第 6–8 页，§4）。本节不补写正文未明确给出的样本总数，也不把模型比较当作组件移除消融。

三个指标需要分开解释：SMA 用编辑距离或余弦相似度衡量生成 Schema 与参考的相似程度；ESR 统计能通过验证并无运行错误地执行的比例；EMR 衡量满足的 SQL 状态断言占全部期望断言的比例。因而 EMR 既不是问答准确率，也不是整条任务全部断言通过的比例（第 7–8 页，§4.1、§4.3）：

$$\mathrm{EMR} = \frac{\sum_i \sum_{a \in A_i} \mathbf{1}[a\text{满足}]}{\sum_i |A_i|}. \tag{1}$$

其中 A_i 为第 i 个实例的断言集合；任务所含断言数可能不同，不能把 ESR 与 EMR 的差值解释为失败任务比例。

表 1 Text2Mem 作者报告的总体指标，原文第 8 页表 2 的 Avg 列

模型	$\mathrm{SMA}_{\mathrm{lev}}$	$\mathrm{SMA}_{\mathrm{cos}}$	ESR	EMR
GPT-4o	0.6776	0.8690	0.9484	0.2532
Qwen2.5-72B-Instruct	0.6195	0.8349	0.9510	0.2535
Claude-4-Sonnet	0.6230	0.8475	0.9594	0.3132
Gemini-2.5-Pro	0.6491	0.8560	0.9479	0.2897
GPT-5	0.6115	0.8396	0.9662	0.2826

表1保留原表 Avg 值，不自行重算为十二种操作的简单均值。总体 ESR 约为 94.79%–96.62%，而 EMR 约为 25.32%–31.32%。可执行性较高与状态效果较弱可以同时发生，最高 ESR 和最高 EMR 也不对应同一模型。两类指标衡量不同对象，不能直接据此断言校验器降低了多少语义错误。

复杂存储与工作流值得进一步分析。原表 2 中五个模型的 Split EMR 均为 0；Merge 仅 GPT-4o 为 0.0465，其余为 0。原表 3 中 GPT-5 的单操作 EMR 为 0.2930，工作流 EMR 为 0.1190，而工作流 ESR 为 1.0000。这些数字支持“运行成功不保证效果正确”的观察，但只适用于该基准设置，不能扩展为模型在任何条件下都无法合并或拆分。

论文未报告独立移除 Validator、Parser 或 Adapter 的组件消融。表 2 是模型/操作比较，表 3 是任务分组分析；附录 F 对指令类型现象提出的复杂度混杂与幸存者偏差解释仍是假设，不能作为控制实验确立的因果结论（第 8、15 页）。

1.4 优缺点与阅读判断

可借鉴之处。统一规范使动作、作用范围与生命周期参数显式化，便于在执行前检查并在执行后追踪；规划与状态效果分开计量，避免用格式正确或无异常返回替代语义验收。对长上下文系统而言，这为定位“检索错目标”“计划错动作”和“后端执行错误”提供了不同检查位置。后者是阅读分析，不是作者证明的误差分解定理。

适用边界与不足。作者明确指出复杂记忆语义缺少正式保证、真实后端覆盖有限，以及未评测端到端长期智能体表现（第 9 页 Limitations）。此外，本文认为断言设计会影响 EMR 的解释力：若只检查权重提高，可能遗漏非目标条目被误改的问题。基准中的单步与工作流结果也不能单独证明迁移到真实个人记忆后仍有同等效果。论文提供了可审计基础，但实际适配成本、长期任务收益及错误传播仍需额外验证。

1.5 与其他论文及本组课设的联系

StructMem 关注事件绑定、综合和问答检索；Text2Mem 关注动作规范与执行后的状态核验。Semantic XPath 可帮助理解目标范围的定位，MetaMem 提醒我们区分已取回证据与正确使用证据，Text2Mem 则提醒我们区分可运行操作与正确状态效果。这是机制联系的阅读比较，不代表相关系统可以不经适配直接拼接，也不能用不同基准的分数比较优劣。

本组主线仍是 StructMem，未实现 Text2Mem 的十二操作后端。A 任务已完成的输入字段白名单、版本哈希和答案隔离，是数据准备与校验工作，不能改称 Text2Mem 复现。可在团队讨论中借鉴“显式输入契约与效果分别检查”的原则；若未来开展操作实验，应固定相同初态、请求和模型，分别报告格式、执行、断言满足和整任务通过情况。此处仅提出讨论方向，不新增小组必做任务，也不预设优化收益。

References:

[1] Wang L, Yang L, Chen B, et al. Text2Mem: A unified memory operation language for memory operating system. In: Proceedings of Liakata M, Moreira V P, Zhang J, et al., editors, Findings of the Association for Computational Linguistics: ACL 2026, San Diego, California, United States: Association for Computational Linguistics, 2026. 2105–2119